% Supabase OAuth + Express/EJS Integration

Author: GitHub Copilot (assistant) Date: 2026-05-24

Overview

This document explains how to integrate Supabase OAuth into an Express + EJS application, describing routes, client/server responsibilities, example code snippets, environment variables, security notes, and a short todo list for implementation.

Architecture & Flow

- User clicks "Sign in with PROVIDER" on an EJS page.
- Server route `GET /auth/provider/:provider` redirects to the Supabase authorize endpoint:
  - `https://<SUPABASE_URL>/auth/v1/authorize?provider={provider}&redirect_to={APP_URL}/auth/callback`
- Supabase performs the provider sign-in and redirects back to `/auth/callback`.
- The OAuth result is delivered to the browser in the URL fragment (hash) by default (e.g., `#access_token=...&refresh_token=...`). The server cannot read the fragment directly—an EJS callback page with a small client-side script must extract the tokens and POST them to the server.
- Server receives the token via POST (e.g., `POST /auth/session`), verifies it using a server-side Supabase client (initialized with the Service Role key or via token introspection), and creates an httpOnly cookie-backed session.

Why a client-side step is needed

Supabase (and many OAuth flows) return tokens in the fragment. Browsers don't send the fragment to the server on redirect, so a client-side script must capture tokens from `location.hash` and send them to the server to establish a server-side session.

Minimal Route Map

- `GET /login` — renders EJS login page with provider links.
- `GET /auth/provider/:provider` — redirects to Supabase authorize URL.
- `GET /auth/callback` — renders an EJS view that extracts tokens and posts them to the server.
- `POST /auth/session` — verifies token server-side, sets httpOnly cookie, redirects to protected area.
- `GET /protected` — middleware checks server cookie and renders protected EJS view.

Example snippets (conceptual)

Server redirect to Supabase (Express):

```
const provider = req.params.provider;
const supabaseHost = process.env.SUPABASE_URL.replace(/^https?
const redirect = `https://${supabaseHost}/auth/v1/authorize?pr
res.redirect(redirect);
```

EJS callback page (client-side script inside `GET /auth/callback` view):

```
<script>
  (async function() {
    const hash = new URLSearchParams(window.location.hash.slic
    const access_token = hash.get('access_token');
    const refresh_token = hash.get('refresh_token');
    if (!access_token) {
      // show error or redirect to login
      window.location = '/login';
      return;
    }
    await fetch('/auth/session', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ access_token, refresh_token })
    });
    window.location = '/dashboard';
  })();
</script>
```

Server session creation (Express):

```
// assume supabaseAdmin is initialized with service role key
const { data, error } = await supabaseAdmin.auth.getUser(acces
if (error) return res.status(401).send('Invalid token');
// set httpOnly cookie
res.cookie('sb_session', access_token, { httpOnly: true, secur
res.sendStatus(204);
```

Middleware to protect routes:

```
async function requireAuth(req, res, next) {
  const token = req.cookies.sb_session;
  if (!token) return res.redirect('/login');
  const { data, error } = await supabaseAdmin.auth.getUser(tok
  if (error || !data?.user) return res.redirect('/login');
  req.user = data.user;
```

```
  next();
}
```

Environment variables (examples)

- Client/browser: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` (public/anon only)
- Server-only: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `APP_URL`
- Provider credentials are configured in the Supabase dashboard (do not expose provider secrets to browsers)

Security notes

- Never expose the Service Role key to the browser.
- Use secure, httpOnly cookies for server sessions.
- Validate tokens server-side for protected requests.
- Consider `state` CSRF protection for custom flows; Supabase handles this for standard authorize flows.
- Use HTTPS and `secure: true` on cookies in production.
- Respect RLS by using the user's JWT for DB requests where you want RLS applied; only use the service role for verification and privileged tasks.

Alternatives

- Fully client-side with `@supabase/supabase-js`: simpler, session is managed in the browser.
- Full server-side OAuth: implement provider OAuth on your server and create Supabase users via Admin APIs — more control, more complexity.

Todo List (current)

- Describe architecture and flow — in-progress
- Map Express routes and EJS views — not-started
- Provide example Express code snippets — not-started
- Detail env vars and security notes — not-started
- Suggest next steps for integration — not-started

End of document.